(Lakhotia and Sterling, 1990) and Mixtus (Sahlin, 1991). An interesting application of partial evaluation is given by Smith (1991), where efficient string-matching programs were developed.

Composition was first discussed in the context of Prolog meta-interpreters in Sterling and Beer (1989) and an informal algorithm was given in Sterling and Lakhotia (1988). A theory is found in Kirschenbaum, Sterling, and Jain (1993).

19 Logic Grammars

A very important application area of Prolog is parsing. In fact, Prolog originated from attempts to use logic to express grammar rules and to formalize the process of parsing. In this chapter, we present the most common logic grammar formalism, definite clause grammars. We show how grammar rules can be considered as a language on top of Prolog, and we apply grammar rules to parse simple English sentences. In Chapter 24, definite clause grammars are used as the parsing component of a simple compiler for a Pascal-like language.

19.1 Definite Clause Grammars

Definite clause grammars arise from adding features of Prolog to context-free grammars. In Section 18.3, we briefly sketched how context-free grammars could be immediately converted to Prolog programs, which parsed the language specified by the context-free grammar. By adding the ability of Prolog to exploit the power of unification and the ability to call builtin predicates, a very powerful parsing formalism is indeed achieved, as we now show.

Consider the context-free grammar for recognizing the language $a^*b^*c^*$, presented in Figure 18.1, with equivalent Prolog program Program 18.8. The Prolog program can be easily enhanced to count the number of symbols that appear in any recognized sequence of a's, b's, and c's. An argument would be added to each predicate constituting the number of symbols found. Arithmetic would be performed to add numbers together. The first clause would become

```
s(As\Xs,N) ←
    a(As\Bs,NA), b(Bs\Cs,NB), c(Cs\Xs,NC), N is NA+NB+NC.
```

The extra argument counting the number of a's, b's, and c's can be added to the grammar rule just as easily, yielding

```
s(N) → a(NA), b(NB), c(NC), N is NA+NB+NC.
```

Adding arguments to nonterminal symbols of context-free grammars, and the ability to call (arbitrary) Prolog predicates, increases their utility and expressive power. Grammars in this new class are called *definite clause grammars*, or *DCG*s. Definite clause grammars are a generalization of context-free grammars that are executable, augmented by the language features of Prolog.

Program 18.9, translating context-free grammars into Prolog programs, can be extended to translate DCGs into Prolog. The extension is posed as Exercise (i) at the end of this section. Throughout this chapter we write DCGs in grammar rule notation, being aware that they can be viewed as Prolog programs. Many Edinburgh Prolog implementations provide support for grammar rules. The operator used for → is `-->`. Grammar rules are expanded automatically into Prolog clauses with two extra arguments added as the last two arguments of the predicate to represent as a difference-list the sequence of tokens or words recognized by the predicate. Braces are used to delimit goals to be called by Prolog directly, which should not have extra arguments added during translation. Grammar rules are not part of Standard Prolog but will probably be incorporated in the future.

Program 19.1 gives a DCG that recognizes the language $a^*b^*c^*$ and also counts the number of letters in the recognized sequence. The enhancement from Figure 18.1 is immediate. To query Program 19.1, consideration must be taken of the two extra arguments that will be added. For example, a suitable query is `s(N,[a,a,b,b,b,c],[ ])?`.

Counting the symbols could, of course, be accomplished by traversing the difference-list of words. However, counting is a simple enhancement to understand, which effectively displays the essence of definite clause grammars. Section 19.3 presents a wider variety of enhancements.

Our next example is a striking one of the increase in expressive power possible using extra arguments and unification. Consider recognizing the language $a^N b^N c^N$, which is not possible with a context-free grammar.


```
s(N) → a(NA), b(NB), c(NC), {N is NA+NB+NC}.
a(N) → [a], a(N1), {N is N1+1}.
a(0) → [ ].
b(N) → [b], b(N1), {N is N1+1}.
b(0) → [ ].
c(N) → [c], c(N1), {N is N1+1}.
c(0) → [ ].
```

Program 19.1 Enhancing the language $a^*b^*c^*$

```
s → a(N), b(N), c(N).
a(N) → [a], a(N1), {N is N1+1}.
a(0) → [ ].
b(N) → [b], b(N1), {N is N1+1}.
b(0) → [ ].
c(N) → [c], c(N1), {N is N1+1}.
c(0) → [ ].
```

Program 19.2 Recognizing the language $a^N b^N c^N$

However, there is a straightforward modification to the grammar given as Program 19.1. All that is necessary is to change the first rule and make the number of a's, b's, and c's the same. The modified program is given as Program 19.2.

In Program 19.2, unification has added context sensitivity and increased the expressive power of DCGs over context-free grammars. DCGs should be regarded as Prolog programs. Indeed, parsing with DCGs is a perfect illustration of Prolog programming using nondeterministic programming and difference-lists. The top-down, left-to-right computation model of Prolog yields a top-down, left-to-right parser.

Definite clause grammars can be used to express general programs. For example, a version of Program 3.15 for `append` with its last two arguments swapped can be written as follows.

```
append([ ]) → [ ].
append([X|Xs]) → [X], append(Xs).
```

Using DCGs for tasks other than parsing is an acquired programming taste.

The grammar for the declarative part of a Pascal program.

```
declarative_part →
    const_declaration, type_declaration,
    var_declaration, procedure_declaration.
```

Constant declarations

```
const_declaration → [ ].
const_declaration →
    [const], const_definition, [;], const_definitions.

const_definitions → [ ].
const_definitions →
    const_definition, [;], const_definitions.

const_definition → identifier, [=], constant.

identifier → [X], {atom(X)}.
constant → [X], {constant(X)}.
```

Type declarations

```
type_declaration → [ ].
type_declaration →
    [type], type_definition, [;], type_definitions.

type_definitions → [ ].
type_definitions → type_definition, [;], type_definitions.

type_definition → identifier, [=], type.

type → ['INTEGER'].
type → ['REAL'].
type → ['BOOLEAN'].
type → ['CHAR'].
```

Variable declarations

```
var_declaration → [ ].
var_declaration →
    [var], var_definition, [;], var_definitions.

var_definitions → [ ].
var_definitions → var_definition, [;], var_definitions.
var_definition → identifiers, [:], type.

identifiers → identifier.
identifiers → identifier, [,], identifiers.
```

Program 19.3 Parsing the declarative part of a Pascal block

Procedure declarations

```
procedure_declaration → [ ].
procedure_declaration → procedure_heading, [;], block.

procedure_heading →
    [procedure], identifier, formal_parameter_part.

formal_parameter_part → [ ].
formal_parameter_part → [(], formal_parameter_section, [)].

formal_parameter_section → formal_parameters.
formal_parameter_section →
    formal_parameters, [;], formal_parameter_section.

formal_parameters → value_parameters.
formal_parameters → variable_parameters.

value_parameters → var_definition.
variable_parameters → [var], var_definition.
```

Program 19.3 (Continued)

We conclude this section with a more substantial example. A DCG is given for parsing the declarative part of a block in a Pascal program. The code does not in fact cover all of Pascal — it is not complete in its definition of types or constants, for example. Extensions to the grammar are posed in the exercises at the end of this section. Parsing the statement part of a Pascal program is illustrated in Chapter 24.

The grammar for the declarative part of a Pascal block is given as Program 19.3. Each grammar rule corresponds closely to the syntax diagram for the corresponding Pascal statement. For example, the syntax diagram for constant declarations is as follows:

```
---> const  -----> Constant Definition -------> ; ------->
            |                                   |
            +---------------<--------------------+
```

The second grammar rule for `const_declaration` in Program 19.3 says exactly the same. A constant declaration is the reserved word `const` followed by a constant definition, handled by the nonterminal symbol `const_definition`; followed by a semicolon; followed by the rest of the constant definition, handled by the nonterminal symbol `const_definitions`. The first rule for `const_declaration` effectively states that the constant declaration is optional. A constant definition is

an identifier followed by =, followed by a constant. The definition for `const_definitions` is recursive, being either empty or another constant definition; followed by a semicolon; followed by the rest of the constant definition.

The remainder of Program 19.3 is similarly easy to understand. It clearly shows the style of writing grammars in Prolog.

Exercises for Section 19.1

(i) Extend Program 18.9 so that it translates definite clause grammars to Prolog as well as context-free grammars.

(ii) Add to Program 19.3 the ability to correctly handle label declarations and function declarations.

(iii) Enhance Program 19.3 to return the list of variables declared in the declarative part.

(iv) Write a program to parse the language of your choice in the style of Program 19.3.

19.2 A Grammar Interpreter

Grammar rules are viewed in the previous section as syntactic sugar for Prolog clauses. This view is supported by Prolog systems with automatic grammar rule translation. There is a second way of viewing grammar rules, namely as a rule language.

This section takes the second view and considers grammar rules as an embedded language on top of Prolog. We consider applying the interpreter techniques of Chapter 17 to grammar rules.

Program 19.4 is an interpreter for grammar rules. The basic relation is `parse(Symbol,Tokens)`, which is true if a sequence of grammar rules can be applied to `Symbol` to reach `Tokens`. The tokens are represented as a difference-list.

The granularity of the DCG interpreter is at the clause reduction level, the same as for the vanilla meta-interpreter, Program 17.5, and the expert system rule interpreter, Program 17.18. Indeed, the code in Program 19.4 is similar to those interpreters. There are four cases, handled by the

parse(Start,Tokens) ←
The sequence of tokens *Tokens* represented as a difference-list can be reached by applying the grammar rules defined by →/2, starting from *Start*.

```
parse(A,Tokens) ←
    nonterminal(A), A → B, parse(B,Tokens).
parse((A,B),Tokens\Xs) ←
    parse(A,Tokens\Tokens1), parse(B,Tokens1\Xs).
parse(A,Tokens) ← terminals(A), connect(A,Tokens).
parse({A},Xs\Xs) ← A.
terminals(Xs) ← See Program 18.9.
connect(Xs,Tokens) ← See Program 18.8.
```

Program 19.4 A definite clause grammar (DCG) interpreter

four clauses for `parse` in Program 19.4. The first rule handles the basic operation of reducing a nonterminal symbol, and the second rule handles conjunctions of symbols. The third rule handles terminal symbols, and the fourth rule covers the ability to handle Prolog predicates by calling them directly using the meta-variable facility.

Observe that the last argument in `parse/2`, the DCG interpreter, is a difference-list. This difference-list can be handled implicitly using grammar rule notation. In other words, Program 19.4 could itself be written as a DCG. This task is posed as Exercise 19.2(i).

Recall that the interpreters of Chapter 17 were enhanced. Similarly, the DCG interpreter, Program 19.4, can be enhanced. Program 19.5 gives a simple enhancement that counts the number of tokens used in parsing. As mentioned before, this particular enhancement could be accomplished directly, but it illustrates how an interpreter can be enhanced.

Comparing Programs 19.1 and 19.5 raises an important issue. Is it better to enhance a grammar by modifying the rules, as in Program 19.1, or to add the extra functionality at the level of the interpreter? The second approach is more modular, but suffers from a lack of efficiency.

Exercises for Section 19.2

(i) Write Program 19.4 as a DCG.

parse(Start,Tokens,N) ←
The sequence of tokens *Tokens*, represented as a difference-list, can be reached by applying the grammar rules defined by →/2, starting from *Start*, and *N* tokens are found.

```
parse(A,Tokens,N) ←
    nonterminal(A), A → B, parse(B,Tokens,N).
parse((A,B),Tokens\Xs,N) ←
    parse(A,Tokens\Tokens1,NA), parse(B,Tokens1\Xs,NB),
    N is NA+NB.
parse(A,Tokens,N) ←
    terminals(A), connect(A,Tokens), length(A,N).
parse({A},Xs\Xs,0) ← A.
terminals(Xs) ← See Program 18.9.
connect(A,Tokens) ← See Program 18.8.
length(Xs,N) ← See Program 8.11.
```

Program 19.5 A DCG interpreter that counts words

(ii) Use the partial reducer, Program 18.3, to specialize the interpreter of Program 19.4 to a particular grammar. For example, Figure 18.1 should be transformed to Program 19.1.

(iii) Enhance Program 19.4 to build a parse tree.

19.3 Application to Natural Language Understanding

An important application area of logic programming has been understanding natural languages. Indeed, the origins of Prolog lie within this application. In this section, it is shown how Prolog, through definite clause grammars, can be applied to natural language processing.

A simple context-free grammar for a small subset of English is given in Program 19.6. The nonterminal symbols are grammatical categories, parts of speech and phrases, and the terminal symbols are English words that can be thought of as the vocabulary. The first rule in Program 19.6 says that a sentence is a noun phrase followed by a verb phrase. The last rule says that *surprise* is a noun. A sample sentence recognized by the grammar is: "The decorated pieplate contains a surprise."

Grammar Rules

```
sentence → noun_phrase, verb_phrase.

noun_phrase → determiner, noun_phrase2.
noun_phrase → noun_phrase2.

noun_phrase2 → adjective, noun_phrase2.
noun_phrase2 → noun.

verb_phrase → verb.
verb_phrase → verb, noun_phrase.
```

Vocabulary

```
determiner → [the].      adjective → [decorated]
determiner → [a].

noun → [pieplate].       verb → [contains].
noun → [surprise].
```

Program 19.6 A DCG context-free grammar

Using the terminology of stepwise enhancement introduced in Chapter 13, we can view a grammar as a skeleton. We proceed to show how useful grammatical features can be added by enhancement. The next two programs are enhancements of Program 19.6. The enhancements, although simple, typify how DCGs can be used for natural language applications. Both programs exploit the power of the logical variable.

The first enhancement is constructing a parse tree for the sentence as it is being parsed. The program is given as Program 19.7. Arguments representing (subparts of) the parse tree must be added to Program 19.6. The enhancement is similar to adding structured arguments to logic programs, as discussed in Section 2.2. The program builds the parse tree top-down, exploiting the power of the logic variable.

The rules in Program 19.7 can be given a declarative reading. For example, consider the rule

```
sentence(sentence(NP,VP)) → noun_phrase(NP), verb_phrase(VP).
```

This states that the parse tree built in recognizing the sentence is a structure `sentence(NP,VP)`, where `NP` is the structure built while recognizing the noun phrase and `VP` is the structure built while recognizing the verb phrase.

```
sentence(sentence(NP,VP)) → noun_phrase(NP), verb_phrase(VP).

noun_phrase(np(D,N)) → determiner(D), noun_phrase2(N).
noun_phrase(np(N)) → noun_phrase2(N).

noun_phrase2(np2(A,N)) → adjective(A), noun_phrase2(N).
noun_phrase2(np2(N)) → noun(N).

verb_phrase(vp(V)) → verb(V).
verb_phrase(vp(V,N)) → verb(V), noun_phrase(N).
```

Vocabulary

```
determiner(det(the)) → [the].
determiner(det(a)) → [a].

noun(noun(pieplate)) → [pieplate].
noun(noun(surprise)) → [surprise].

adjective(adj(decorated)) → [decorated].
verb(verb(contains)) → [contains].
```

Program 19.7 A DCG computing a parse tree

The next enhancement concerns subject/object number agreement. Suppose we wanted our grammar also to parse the sentence "The decorated pieplates contain a surprise." A simplistic way of handling plural forms of nouns and verbs, sufficient for the purposes of this book, is to treat different forms as separate words. We augment the vocabulary by adding the facts

```
noun(noun(pieplates)) → [pieplates].
verb(verb(contain)) → [contain].
```

The new program would parse "The decorated pieplates contain a surprise" but unfortunately would also parse "The decorated pieplates contains a surprise." There is no insistence that noun and verb must both be singular, or both be plural.

Number agreement can be enforced by adding an argument to the parts of speech that must be the same. The argument indicates whether the part of speech is singular or plural. Consider the grammar rule

```
sentence(sentence(NP,VP)) →
    noun_phrase(NP,Num), verb_phrase(VP,Num).
```

The rule insists that both the noun phrase, which is the subject of the sentence, and the verb phrase, which is the object of the sentence, have


```
sentence(sentence(NP,VP)) →
    noun_phrase(NP,Num), verb_phrase(VP,Num).

noun_phrase(np(D,N),Num) →
    determiner(D,Num), noun_phrase2(N,Num).
noun_phrase(np(N),Num) → noun_phrase2(N,Num).

noun_phrase2(np2(A,N),Num) →
    adjective(A), noun_phrase2(N,Num).
noun_phrase2(np2(N),Num) → noun(N,Num).

verb_phrase(vp(V),Num) → verb(V,Num).
verb_phrase(vp(V,N),Num) →
    verb(V,Num), noun_phrase(N,Num1).
```

Vocabulary

```
determiner(det(the),Num) → [the].
determiner(det(a),singular) → [a].

noun(noun(pieplate),singular) → [pieplate].
noun(noun(pieplates),plural) → [pieplates].
noun(noun(surprise),singular) → [surprise].
noun(noun(surprises),plural) → [surprises].

adjective(adj(decorated)) → [decorated].

verb(verb(contains),singular) → [contains].
verb(verb(contain),plural) → [contain].
```

Program 19.8 A DCG with subject/object number agreement

the same number, singular or plural. The agreement is indicated by the sharing of the variable Num. Expressing subject/object number agreement is context-dependent information, which is clearly beyond the scope of context-free grammars.

Program 19.8 is an extension of Program 19.7 that handles number agreement correctly. Noun phrases and verb phrases must have the same number, singular or plural. Similarly, the determiners and nouns in a noun phrase must agree in number. The vocabulary is extended to indicate which words are singular and which plural. Where number is unimportant, for example, with adjectives, it can be ignored, and no extra argument is given. The determiner *the* can be either singular or plural. This is handled by leaving the argument indicating number uninstantiated.

The next example of a DCG uses another Prolog feature, the ability to refer to arbitrary Prolog goals in the body of a rule. Program 19.9 is a grammar for recognizing numbers written in English up to, but not including, 1,000. The value of the number recognized is calculated using the arithmetic facilities of Prolog.

The basic relation is `number(N)`, where `N` is the numerical value of the number being recognized. According to the grammar specified by the program, a number is zero or a number `N` of at most three digits, the relation `xxx(N)`. Similarly `xx(N)` represents a number `N` of at most two digits, and the predicates `rest_xxx` and `rest_xx` denote the rest of a number of three or two digits, respectively, after the leading digit has been removed. The predicates `digit`, `teen`, and `tens` recognize, respectively, single digits, the numbers 10 to 19 inclusive, and the multiples of ten from 20 to 90 inclusive.

A sample rule from the grammar is

```
xxx(N) →
    digit(D), [hundred], rest_xxx(N1), {N is D*100+N1}.
```

This says that a three-digit number `N` must first be a digit with value `D`, followed by the word *hundred* followed by the rest of the number, which will have value `N1`. The value for the whole number `N` is obtained by multiplying `D` by 100 and adding `N1`.

DCGs inherit another feature from logic programming, the ability to be used backward. Program 19.9 can be used to generate the written representation of a given number up to, but not including, 1,000. In technical terms, the grammar generates as well as accepts. The behavior in so doing is classic generate-and-test. All the legal numbers of the grammar are generated one by one and tested to see whether they have the correct value, until the actual number posed is reached. This feature is a curiosity rather than an efficient means of writing numbers.

The generative feature of DCGs is not generally useful. Many grammars have recursive rules. For example, the rule in Program 19.6 defining a `noun_phrase2` as an adjective followed by a `noun_phrase2` is recursive. Using recursively defined grammars for generation results in a nonterminating computation. In the grammar of Program 19.7, noun phrases with arbitrarily many adjectives are produced before the verb phrase is considered.


```
number(0) → [zero].
number(N) → xxx(N).

xxx(N) →
    digit(D), [hundred], rest_xxx(N1), {N is D*100+N1}.
xxx(N) → xx(N).

rest_xxx(0) → [ ].
rest_xxx(N) → [and], xx(N).

xx(N) → digit(N).
xx(N) → teen(N).
xx(N) → tens(T), rest_xx(N1), {N is T+N1}.

rest_xx(0) → [ ].
rest_xx(N) → digit(N).

digit(1) → [one].        teen(10) → [ten].
digit(2) → [two].        teen(11) → [eleven].
digit(3) → [three].      teen(12) → [twelve].
digit(4) → [four].       teen(13) → [thirteen].
digit(5) → [five].       teen(14) → [fourteen].
digit(6) → [six].        teen(15) → [fifteen].
digit(7) → [seven].      teen(16) → [sixteen].
digit(8) → [eight].      teen(17) → [seventeen].
digit(9) → [nine].       teen(18) → [eighteen].
                           teen(19) → [nineteen].

tens(20) → [twenty].
tens(30) → [thirty].
tens(40) → [forty].
tens(50) → [fifty].
tens(60) → [sixty].
tens(70) → [seventy].
tens(80) → [eighty].
tens(90) → [ninety].
```

Program 19.9 A DCG for recognizing numbers

Exercises for Section 19.3

(i) Write a simple grammar for French that illustrates gender agreement.

(ii) Extend and modify Program 19.9 for parsing numbers so that it covers all numbers less than 1 million. Don't forget to include things like "thirty-five hundred" and to not include "thirty hundred."

19.4 Background

Prolog was connected to parsing right from its very beginning. As mentioned before, the Prolog language grew out of Colmerauer's interest in parsing, and his experience with developing Q-systems (Colmerauer, 1973). The implementors of Edinburgh Prolog were also keen on natural language processing and wrote one of the more detailed accounts of definite clause grammars (Pereira and Warren, 1980). This paper gives a good discussion of the advantages of DCGs as a parsing formalism in comparison with augmented transition networks (ATNs).

The examples of using DCGs for parsing languages in Section 19.1 were adapted from notes from a tutorial on natural language analysis given by Lynette Hirschman at the Symposium on Logic Programming in San Francisco in 1987. The DCG interpreter of Section 19.2 is adapted from Pereira and Shieber (1987).

Even though the control structure of Prolog matches directly that of recursive-descent, top-down parsers, other parsing algorithms can also be implemented in it quite easily. For example, Matsumoto et al. (1986) describes a bottom-up parser in Prolog.

The grammar in Program 19.3 is taken from Appendix 1 of Findlay and Watt (1985). The grammar in Program 19.6 is taken from Winograd's (1983) book on computational linguistics.

For further reading on logic grammars, refer to Pereira and Shieber (1987) and Abramson and Dahl (1989).

20 Search Techniques

In this chapter, we show programs encapsulating classic AI search techniques. The first section discusses state-transition frameworks for solving problems formulated in terms of a state-space graph. The second discusses the minimax algorithm with alpha-beta pruning for searching game trees.

20.1 Searching State-Space Graphs

State-space graphs are used to represent problems. Nodes of the graph are states of the problem. An edge exists between nodes if there is a transition rule, also called a *move*, transforming one state into the next. Solving the problem means finding a path from a given initial state to a desired solution state by applying a sequence of transition rules.

Program 20.1 is a framework for solving problems by searching their state-space graphs, using depth-first search as described in Section 14.2.

No commitment has been made to the representation of states. The moves are specified by a binary predicate `move(State,Move)`, where `Move` is a move applicable to `State`. The predicate `update(State,Move, State1)` finds the state `State1` reached by applying the move `Move` to state `State`. It is often easier to combine the `move` and `update` procedures. We keep them separate here to make knowledge more explicit and to retain flexibility and modularity, possibly at the expense of performance.

The validity of possible moves is checked by the predicate `legal (State)`, which checks if the problem state `State` satisfies the constraints of the problem. The program keeps a history of the states visited